2025-2026 Project: Tennis Database Prediction

Giacomo Tocchetto 906262

Introduction

In this year's project I was asked to make and evaluate two prediction methods on a tennis dataset (made by www.tennis-data.co.uk), fine tuning their parameters. The purpose of the project is to predict the winner of a match.

For this project I have decided to utilize 5 datasets: from 2025 to 2021. The datasets contained statistics of all the matches in the top tournaments in the world such as: ATP, Grand Slam and Masters.

Each dataset had a lot of statistics like match metadata, player statistics, match outcomes and financial betting odds.

Data Preparation

To get the most out of the prediction model that is being used it is necessary to feed it clean data. It is also necessary to avoid data leakage, which is the concept of feeding the model data it should not be seeing. For example, training it with the results of the match would be data leakage, because we are showing the model data that is known after the target is known.

For this reason, I decided to drop all the match statistics like the Sets results. I have also decided to drop other features that I did not find useful to determine a winner, such as: Location, Name of the Tournament or the ATP number.

I have also made the critical choice to drop all of the Betting Odds, there are two reasons behind this, one being that these odds were accounting a big chunk of the model's feature importance, the other being that since casinos have their own prediction models, that are most likely trained on a massive amount of data and engineered features, my model was just learning how to mimic the casinos rather than predicting the physical sport.

A critical structural issue in the dataset is how statistics are labeled. Every match is recorded from the perspective of the attributes 'Winner' and 'Loser'. If kept like this, statistics would lead to severe Data Leakage, as the way columns are structured reveals the outcome. For this reason, the whole dataset had to be restructured into an unbiased binary classification problem.

I used a randomization mask (`mask=rng.integers(0, 2, size=len(df))`) to create the neutral attributes 'Player_1' and 'Player_2'. Consequently the 'Winner' and 'Loser' attributes have been dropped from the dataset.

To give the model its "ground truth" I created the attribute 'P1_Won': this value simply dictates whether the player classified as 'Player_1' has won the match. I have structured it as a binary value: 1=Won, 0=Lost. This forces the model to evaluate each feature rather than relying on the dataset structure

Finally, the corresponding player statistics were reworked to be all neutral (e.g. 'WRank' became 'Player_1_Rank').

A crucial methodological decision for this prediction model was the use of the classic parameter '`cv=x`' which is essential for hyperparameter-tuning methods like `GridSearchCV`. CV stands for Cross-Validation, which consists in taking the data and splitting it into x folds, training the model on x-1 folds and evaluating it on the holdout fold. Cross-Validation repeats the process x times and averages the scores to determine the optimal parameter combination.

However, using standard Cross-Validation in prediction model where chronological order matters, introduces severe Data Leakage, also called **Look-Ahead Bias**. The reason for this is that the splits are random, and this would allow the method to train on data from 2025 and validate it on data from 2024.

To prevent this, **Time Series Cross-Validation** was introduced: it respects the temporal order of the dataset and forces the model to evaluate only on future datasets.

Feature Engineering

It is known that if you want to build a prediction model, you must engineer your own features, because raw data itself is not enough.

I will now explain all the engineered features that take part in this project.

Let's start from the simplest, gradually moving up to the most complicated:

- `Delta_Rank, Delta_Points`**:** I have engineered those statistics by simply computing the difference between the '`Player_1`' and '`Player_2`' value. This is a common technique in machine learning, because it helps the model to directly isolate changes, learn faster and generalize better.
- `Round/Surface`**:** To give the model more knowledge about each match, I turned the "Round" feature into an integer value, the original feature ranged between the lowest and more accessible rounds to the absolute final, I gave a crescent value from lowest to highest, ranging from 0 to 10.
- `P1/2_sets_last_7d`**:** This feature is made to express each player's fatigue; the core idea is to compute not just how many matches the player had in the last week but to accumulate the amount of sets he had to sustain for each match. This is done with a two *pandas* functions: `rolling()` and `sum().`
- `P1/2_winrate_30d`**:** This feature is made to express whether the player has been in a winning or losing streak in the past month. It is coded very similarly to the above one, changing `sum()` to `mean(),` in this way the function sees a set of 1s and 0s, respectively meaning victory and loss, the function computes a mean that shows the win percentage in the last 30 days.
- `H2H_ratio`**:** this feature represents the ratio between Player_1's wins and the total number of previous meetings between the two players (P1_h2h_wins / total h2h matches). If the two players have never met before, the value defaults to 0.5, representing a neutral prior with no historical signal.
- `Delta_clay_wr/Delta_hard_wr/Delta_grass_wr`**:** these features are very similar to each other because they represent the difference between the two players' win rates on each surface, which also have been calculated using the `mean()` function.

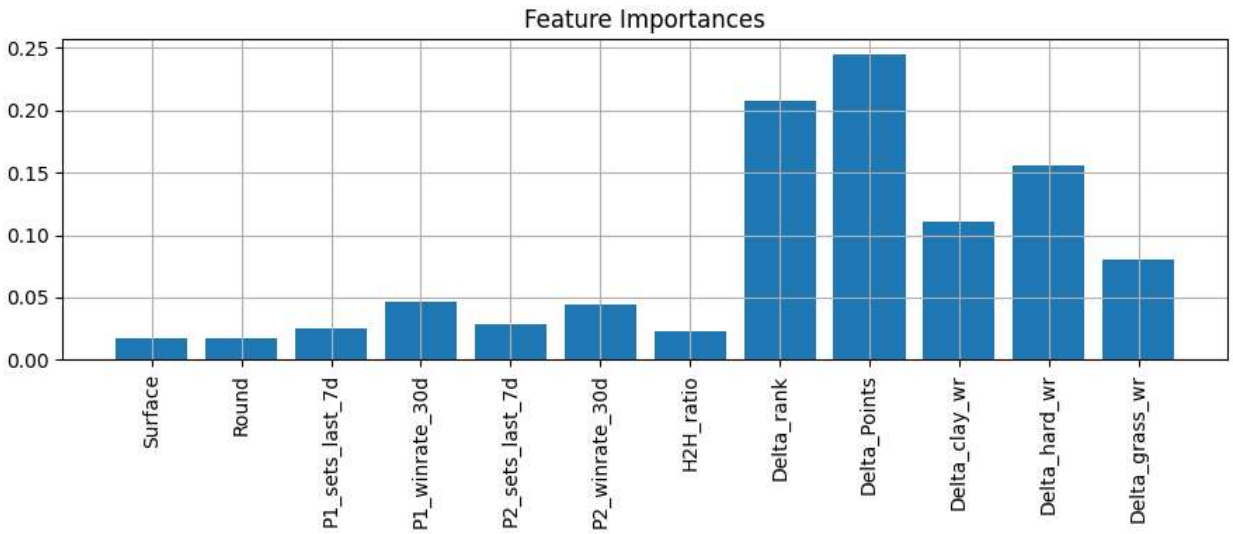
Models and Hyper-Parameter Tuning

For this project two models were evaluated on the same dataset, and each one's hyper-parameters were tuned to get the most of it:

1. **Random Forest**: Random Forest builds a set of different trees using Feature Bagging, Bootstrapping and Bagging. With this dataset it worked well, achieving good accuracy from the beginning. For this exact reason, the hyper-parameter tuning barely even helped, rising the accuracy only by ~0.30%. `GridSearchCV` was used in combination with Time Series Cross-Validation to respect the temporal order of the dataset and prevent Look-Ahead Bias. The scoring metric that seemed to be working best was neg_log_loss. This adjustment forced the algorithm to optimize the quality of its probability estimates. Key parameters such as `max_depth, min_samples_leaf, and max_features` were fine-tuned to balance tree complexity, prevent overfitting, and ensure diverse feature selection across the entire forest. RFECV was also used but it did not provide any help.
2. **XGBoost:** XGBoost is very similar to Random Forest, the only difference is that trees in this model use gradient boosting, which means that XGBoost builds these trees sequentially and each new tree is specifically optimized to correct the residual errors made by the previous ones. While this advanced architecture theoretically offers higher performance, the model on its initial baseline performed ~3% worse than Random Forest. This shows how the model is sensitive to its hyper-parameters, because tuning them by applying `RandomizedSearchCV` on learning rate, tree depth, and column sampling, it was able to reach an accuracy of 64.12%.

Feature Importances

To better work on investigating feature importances, pandas was used to create a bar chart, visualizing each instance.



In this plot we can easily visualize the impact of each independent feature contribution to the Random Forest's splitting criteria. The players' point and rank difference are the most impacting, this is because a player's rank is a direct mathematical derivative of their points, they are highly collinear since they represent. It might sound common, but this is how matches go for most times, and we get the confirmation of this thesis by knowing that these two features are the most important for the Random Forest algorithm.

Beyond the baseline rankings, the prominence of the Surface win rate attributes (`Delta_hard_wr, Delta_clay_wr, Delta_grass_wr`) is a critical finding. Ranking as the third most important feature, it demonstrates that the algorithm does not treat all matches equally. Instead, the model successfully utilizes

those win rates as a primary contextual filter, fundamentally altering its predictive logic based on whether the match is played on clay, hard, or grass courts.

This is because in tennis surface isn't just simple data: it dictates how high, how fast and how far the ball bounces this means that it dictates entire rhythm of the game, a consequence from this is that for each Surface, different physical skills and strategies are needed so a player that performs excellently on Grass, might be average on Hard.

Following this, comes the momentum feature (`P1_winrate_30d`) proving that recent form provides the crucial final adjustments to the baseline rank predictions.

Conversely, features such as Fatigue (`sets_last_7d`), tournament Round, and Head-to-Head records (`H2H_ratio`) provided minimal influence on the model's splitting criteria. The low impact of the Head-to-Head metric is primarily due to **data sparsity**: in professional tennis, most players rarely face each other more than a few times, resulting in a lack of statistically significant historical matchups for the algorithm to leverage. Furthermore, the low importance of fatigue suggests that at the elite ATP level, player conditioning is good enough that short-term match volume does not highly affect each player's performance. Finally, the tournament Round acts just as environmental metadata, because both athletes are competing in the exact same round of the tournament, it provides almost no comparative mathematical advantage to the model.

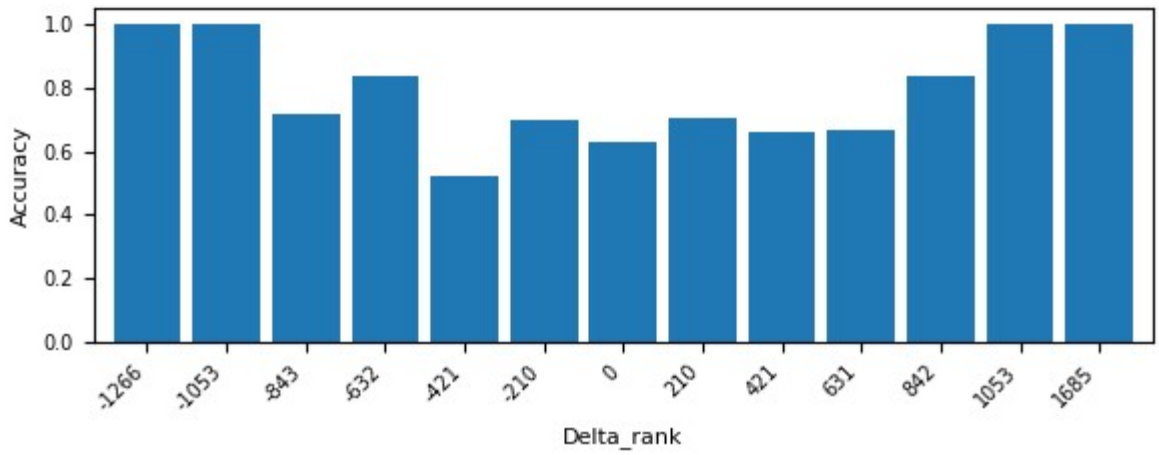
Feature Investigation

Some tests have been run on the most correct and wrong predictions; to identify peculiarities, the mean of each feature was computed, split by Correct and Wrong predictions. The difference between these two groups was then calculated and scaled to determine the Standardized Mean Difference (SMD). Here are a few columns:

	Wrong Prediction	Correct Prediction	Difference	SMD
`P2_winrate_30d`	0.470463	0.515777	0.045314	0.140669
`P2_sets_last_7d`	2.868644	3.171374	0.302730	0.095913
`Delta_Points`	-6.966949	-19.419339	-12.452390	-0.005187
`Delta_rank`	-0.594068	1.644806	2.238874	0.016464

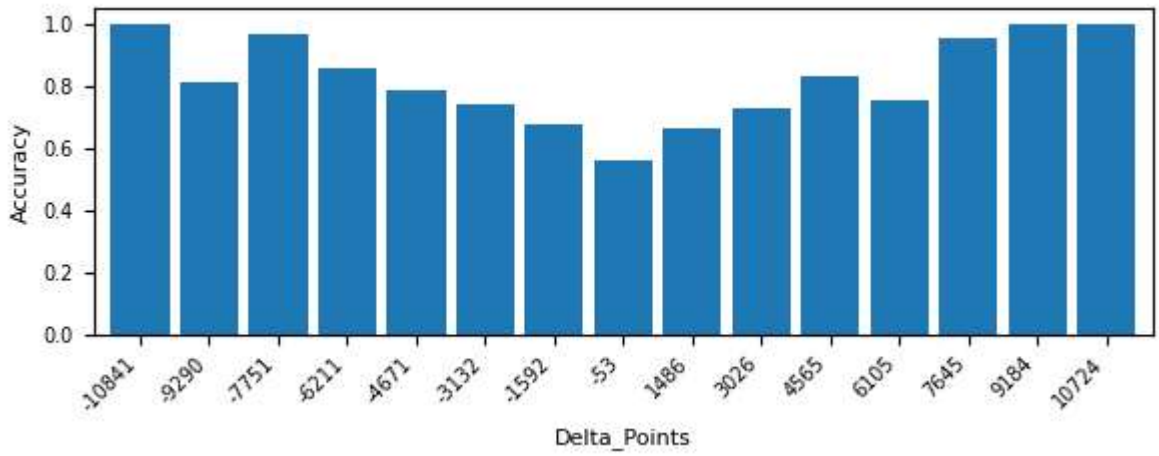
This table shows that what really separates hits from misses are the fatigue and momentum features. In fact, the wrong prediction tends to have lower recent winrate and lower recent match activity, they are also the statistics that stand out more in the Standardized Mean Difference. The two most relevant features for the splitting criteria have also been reported: they show an SMD close to 0, suggesting that they play no role in distinguishing correct from incorrect predictions. However, SMD is very useful to capture different values in the mean, but it can miss some patterns that are symmetrical.

For this reason, `Delta_rank` was split into chunks, for example -900 to -700, and the accuracy was checked for each chunk, revealing a contradiction of the SMD: accuracy is highest at the extremes of `Delta_rank`, which is where a player is ranked way above the other, and drops to 60-70% near zero, where the two players are closely matched. In other words, `Delta_rank` and `Delta_Points` do drive errors — but the failure mode is specific to close matchups, not a general weakness of the feature, and a mean-based test like SMD cannot detect it.



As we can see, the bars roughly create a U shape, highest at the extremes, lower in the middle.

The same thing was done with the `Delta_Points` feature, it showed the same U-shaped chart, but this time it was even more defined. This suggests that rank position is a noisier proxy for the true skill gap than accumulated ranking points, and that the model's errors are best explained by the size of the points gap, not the rank gap: matches between two closely pointed players are near a gamble that the model cannot reliably call, while points advantage still holds a strong signal.



To better prove this an excel file was created, with the objective to show the actual instances, the best example to show is a match with a `Delta_rank = -26`, and `Delta_Points = 5825`, the model predicted the win of Player 1 with 88% confidence, completely failing.

Another instance that is opposite to the one just mentioned: in another match the `Delta_rank` sits at -308, a quite uncertain value, as shown in the above chart but the `Delta_Points` is very big, to be precise it equals to 8678, a value where the model is quite confident, reaching an accuracy of ~100%, the model, in fact, completely nails this prediction with a 0.936 prediction that the winner is the exact one.

A secondary analysis using XGBoost confirmed these findings; despite the different tree-building architecture, XGBoost relied on the exact same core features (`Delta_Points, Delta_rank, and Surface Win Rates`), proving these are the most robust predictors of a match outcome.